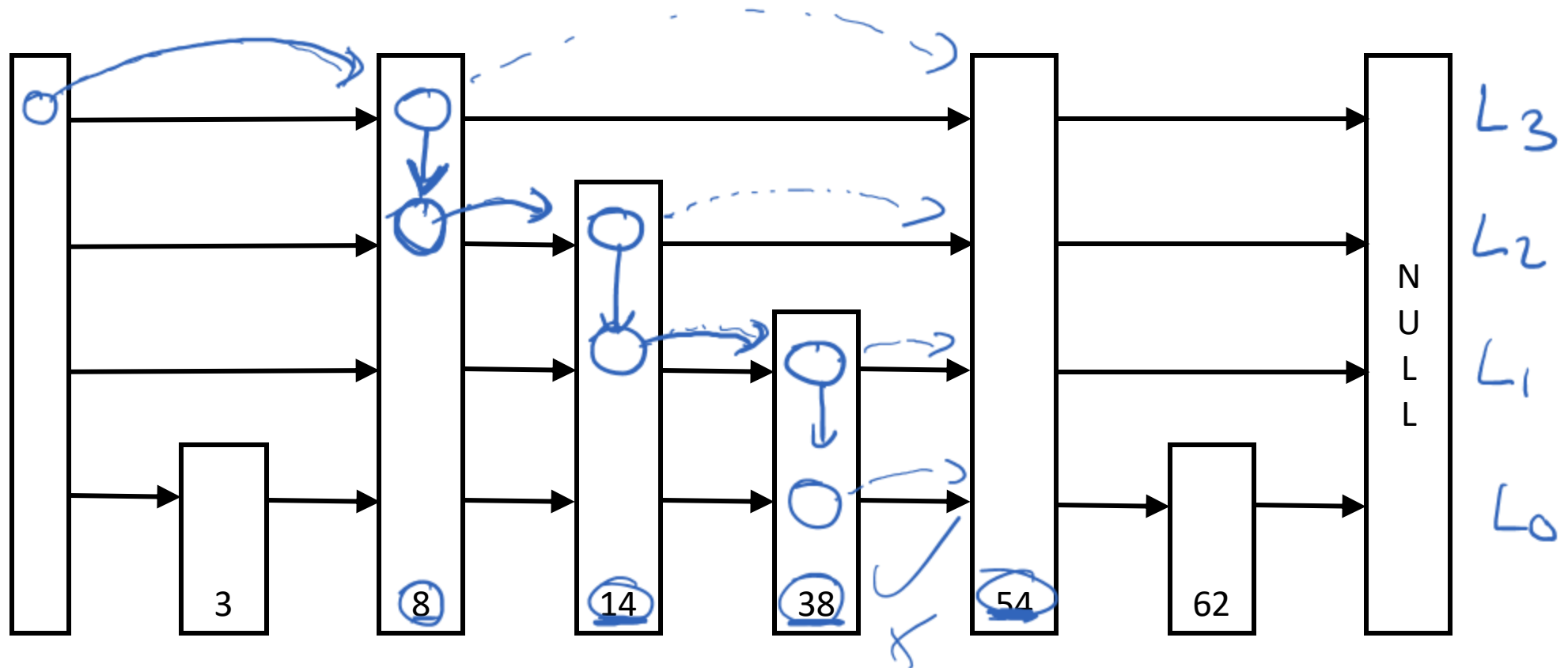


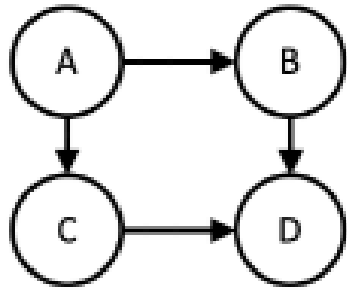
What numbers will the following code print to the terminal?

```
Stream.of(10, 7, 4, 2, 5, 11) → 10, 7, 4, 2, 5, 11  
  .filter( (item) -> item % 2 == 1 ) → 7, 5, 11  
  .map( (item) -> item / 2 ) → 3, 2, 5  
  .forEach( (item) -> System.out.print(item + " ") ); → 3 2 5
```

We want to look up 40 in the following skip list. What keys will we compare 40 to during the search process until we know that it is not in the list?



Which of the following adjacency matrices represents the graph in the diagram? The row labels of the matrices represent the source nodes of an edge and the column labels represent the target nodes.



✓ →

A.

	A	B	C	D
A	0	1	1	0
B	0	0	0	1
C	0	0	0	1
D	0	0	0	0

→

~~B.~~

	A	B	C	D
A	1	1	1	0
B	0	1	0	1
C	0	0	1	1
D	1	0	0	1

→

~~C.~~

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

→

~~D.~~

	A	B	C	D
A	0	1	1	1
B	0	0	0	1
C	0	0	0	1
D	0	0	0	0

When we click on the Label within the following JavaFX application, what will be printed out to the console? The main method and imports are omitted from the code.

```
public class JavaFXApp extends Application {
```

```
    @Override
```

```
    public void start(Stage stage){
```

```
        → Label label = new Label("some label");
```

```
        → HBox hbox = new HBox(label);
```

```
        → Button btn = new Button("some button");
```

```
        → BorderPane bp = new BorderPane();
```

```
        → bp.setTop(hbox);
```

```
        → bp.setBottom(btn);
```

```
        → Scene scene = new Scene(bp, 320, 240);
```

```
        → stage.setScene(scene);
```

```
        → stage.setTitle("JavaFXApp");
```

```
        → scene.addEventHandler(MouseEvent.MOUSE_CLICKED, (event) → System.out.println("A"));
```

```
        → bp.addEventHandler(MouseEvent.MOUSE_CLICKED, (event) → System.out.println("B"));
```

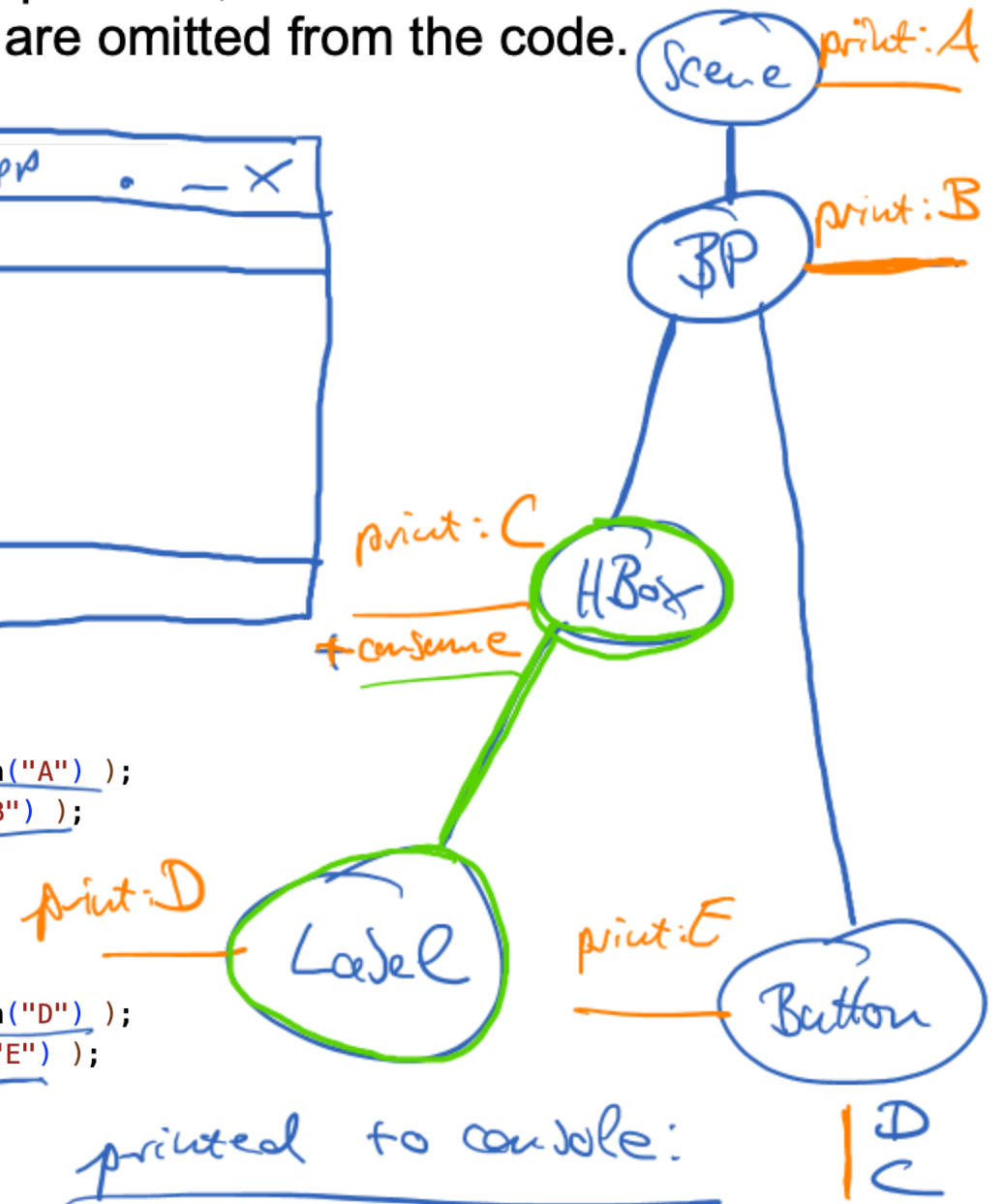
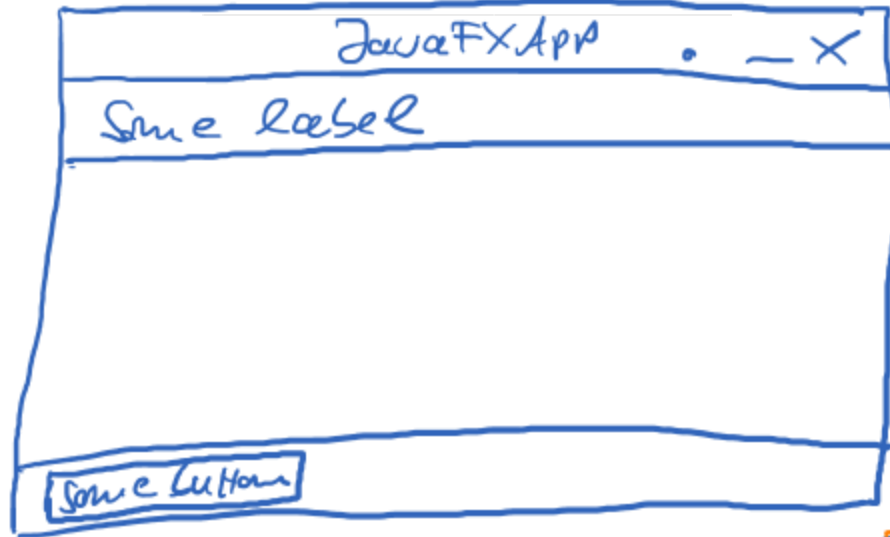
```
        → hbox.addEventHandler(MouseEvent.MOUSE_CLICKED, (event) → {  
            System.out.println("C");  
            event.consume();  
        });
```

```
        → label.addEventHandler(MouseEvent.MOUSE_CLICKED, (event) → System.out.println("D"));
```

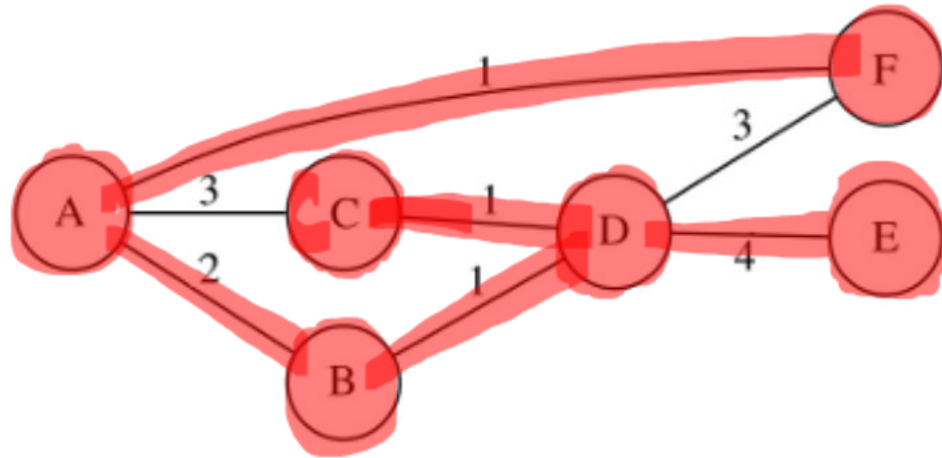
```
        → btn.addEventHandler(MouseEvent.MOUSE_CLICKED, (event) → System.out.println("E"));
```

```
        → stage.show();
```

```
    }
```



Trace through Kruskal's Algorithm on the following graph. If you need a starting node for the algorithm, use node A. Which edges are added to the spanning tree in which order during this trace?

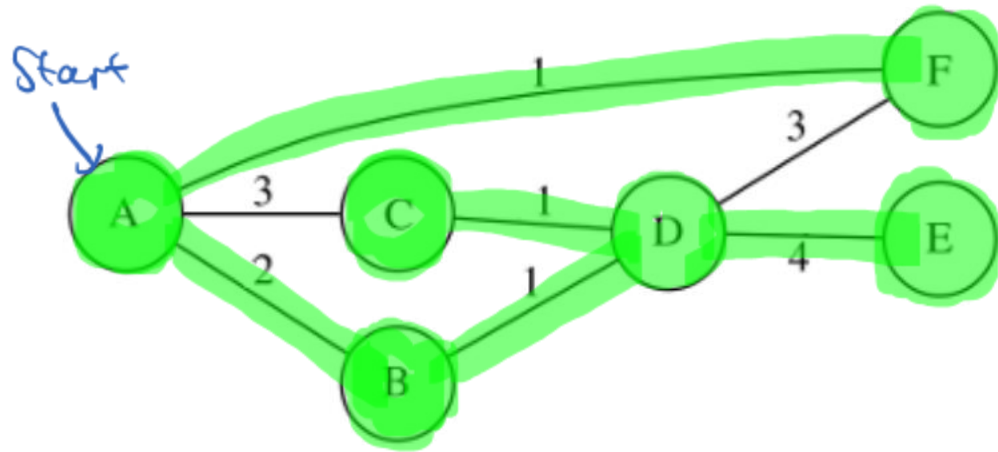


Sorted list of edges:

~~AF:1, CD:1, BD:1, AB:2, AC:3, DF:3, DE:4~~

Selected list: AF:1, CD:1, BD:1, AB:2, DE:4

Trace through Prim's Algorithm on the following graph. If you need a starting node for the algorithm, use node A. Which edges are added to the spanning tree in which order during this trace?



p.p.:

~~AF:1~~, ~~AB:2~~, ~~AC:3~~, ~~FD:3~~, ~~BD:1~~, ~~DC:1~~, ~~DE:4~~

Selected edges: AF:1, AB:2, BD:1, DC:1, DE:4